

Security Testing Report

Web Application Security Assessment using DVWA

1. Introduction

This report presents the results of a web application security assessment performed on the **Damn Vulnerable Web Application (DVWA)**. The purpose of this assessment was to identify common web vulnerabilities and demonstrate how they can be exploited in a controlled environment. The testing was conducted using **Kali Linux** with tools such as **Burp Suite** and manual testing techniques.

2. Testing Environment

Attacker Machine: Kali Linux

Target Application: DVWA (Damn Vulnerable Web Application)

Web Server: Apache

Database: MariaDB

Tools Used:

- Burp Suite
- Browser Developer Tools
- cURL
- Manual testing

DVWA security level was configured to **LOW** to demonstrate vulnerabilities clearly.

3. Vulnerability Assessment

3.1 SQL Injection

Description:

SQL Injection occurs when an attacker inserts malicious SQL queries into input fields to manipulate the backend database.

Test Performed:

The following payload was entered in the User ID field:

1' OR '1'='1

Result:

The application returned multiple user records from the database.

Impact:

Attackers can:

- Access sensitive database information
- Bypass authentication mechanisms
- Extract or modify data

3.2 Stored Cross-Site Scripting (Stored XSS)

Description:

Stored XSS allows attackers to permanently store malicious scripts inside a web application.

Test Performed:

The following script was entered in the message field:

```
<script>alert('Stored XSS')</script>
```

Result:

The script executed when the page loaded, generating a JavaScript alert.

Impact:

Attackers could:

- Steal session cookies
- Hijack user accounts
- Execute malicious scripts in users' browsers

3.3 Reflected Cross-Site Scripting (Reflected XSS)

Description:

Reflected XSS occurs when user input is immediately returned by the server without proper validation.

Test Performed:

```
<script>alert('Reflected XSS')</script>
```

Result:

The script executed in the browser.

Impact:

Attackers could craft malicious links to:

- Steal session tokens
- Redirect users to malicious sites

3.4 File Inclusion (Local File Inclusion)

Description:

File Inclusion vulnerabilities allow attackers to include unauthorized files from the server.

Test Performed:

The following payload was used in the URL parameter:

```
?page=../../../../../var/www/html/DVWA/index.php
```

Result:

The application included another server file.

Impact:

Attackers could:

- Access sensitive system files
 - Execute malicious code
 - Gain unauthorized information from the server
-

3.5 Cross-Site Request Forgery (CSRF)

Description:

CSRF attacks force authenticated users to perform unintended actions.

Test Performed:

The password change function was tested without verifying request authenticity.

Result:

The password was changed successfully without additional verification.

Impact:

Attackers could:

- Change user passwords
 - Perform actions on behalf of authenticated users
-

3.6 HTTP Request Interception using Burp Suite

Description:

Burp Suite was used to intercept and analyze HTTP requests between the client and server.

Observation:

The login request was captured showing parameters such as:

username=admin&password=password

Impact:

Burp Suite allows security testers to:

- Intercept HTTP requests
 - Modify parameters
 - Identify vulnerabilities in web applications.
-

3.7 Web Security Headers Analysis

Security headers were analyzed using the following command:

```
curl -I http://localhost/DVWA
```

Observation:

Important security headers were missing, including:

- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options

Impact:

Missing security headers can expose applications to attacks such as:

- Clickjacking
- Cross-site scripting
- MIME-type attacks

4. Conclusion

The security assessment demonstrated several common web application vulnerabilities within the DVWA environment. The testing confirmed the presence of SQL Injection, Cross-Site Scripting, File Inclusion, and CSRF vulnerabilities. These vulnerabilities highlight the importance of proper input validation, secure coding practices, and the implementation of security controls such as HTTP security headers.

Conducting security assessments helps organizations identify weaknesses and strengthen their web application security posture.